

Lecture 12 — The Gradebook Application

Hitting the wall in a design, and refactoring your way out

Amir Farbin

Where we left off

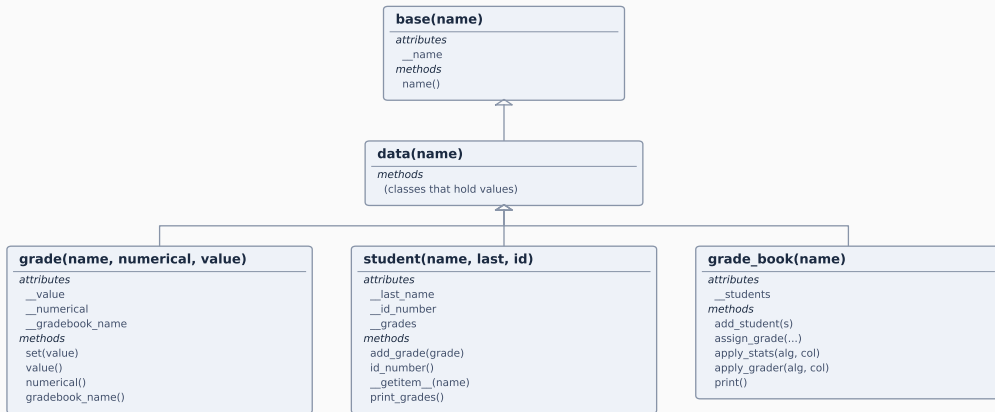
The story so far

- Lecture 11: a messy file → clean data → grading schemes → a first design
- Two kinds of class, kept apart on purpose:
 - **data**: grade, student, grade_book — they hold values
 - **algorithms**: the calculators — they hold only settings, and do work
- Today: the design is used in anger, and it **pushes back**

→ cells 0–2

Last lecture's classes, rebuilt

Data classes: they hold values



Every class carries a name, so an instance can be found and reported on.

Same code, quickly

- `base / data / alg`, then `grade`, `student`
- `grade_calculator`, `statistics_calculator`, `summary_calculator`
- `grade_book` with `apply_stats`, `apply_grader`, `apply_summary`
- Nothing new yet — we need it all on the table before we can criticise it

→ cells 4, 6, 8, 10, 13, 15, 17

Loading it up

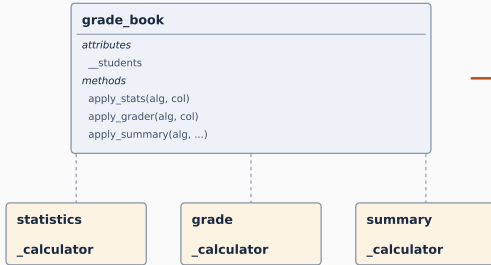
- `build_grade_book` — rows become students, columns become grades
- It has to redo Lecture 11's cleaning: the data comes in as text again
 - a lesson in itself: the fix lived in a notebook, not in the code
- Run a grader, run a summer, print the students

→ cells 24, 25, 27, 29, 30

The wall

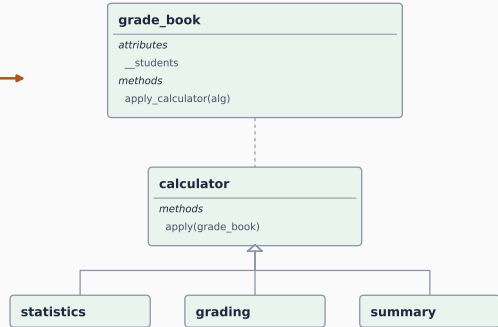
Three kinds, three methods

Lecture 11: three kinds, three methods



the caller has to know which kind it holds

Lecture 12: one interface



the grade book no longer cares

What's actually wrong

- `apply_stats`, `apply_grader`, `apply_summary` — nearly the same code three times
- They differ **only because we decided they should**
- Want to curve *and then* normalize? There's no way to compose them
- Every new kind of calculation means a new method on `grade_book`
- The caller has to know which kind it's holding — that kills polymorphism

The rule being broken

- The class should decide what it does; the caller shouldn't have to
- One name, many implementations — that's the whole point of a base class
- We had it in Lecture 9 with `Shape.paint`, and lost it here
- Fix: make every algorithm look the same from outside

The refactor

A blackboard

- Let the grade book hold **arbitrary results**, not just students' grades
- `__getitem__` / `__setitem__` — the book behaves like a dictionary
- One algorithm writes Lab 2 stats; the next one reads it
- This is how a particle-physics experiment's analysis software is organised, and most data pipelines besides

→ cell 39

One interface

- `calculator.apply(grade_book, **kwargs)`, and raise `NotImplementedError` in the base
- `grade_book.apply_calculator(calc)` — one method, forever
- The book no longer knows or cares what kind of calculation it was handed
- Adding a calculation adds a class. Nothing else changes.

→ cells 37, 43, 46, 47

Migrating the old ones

- Each old algorithm gets rewritten to the new interface, one at a time
- This is what real migration work looks like: unglamorous, and most of the job
- Keep the old copies until the new ones give the same answers
- Careful with generated rewrites: if you can't explain the diff, you can't ship it

→ cells 56, 57, 60, 61, 63, 64, 65

Running them in sequence

- Stats → curve → summary, each reading what the last one left behind
- The order is now a *choice the user makes*, not something baked into the class
- Compose freely — the thing we couldn't do half an hour ago

→ cells 66, 67, 68, 73

Why `lambda` here?

- We hand the book a *way to make* the calculator, not the calculator itself
- Nothing is built until it's needed — lazy instantiation
- Same idea as Lecture 5's generators, and Lecture 10's lazy matrix product

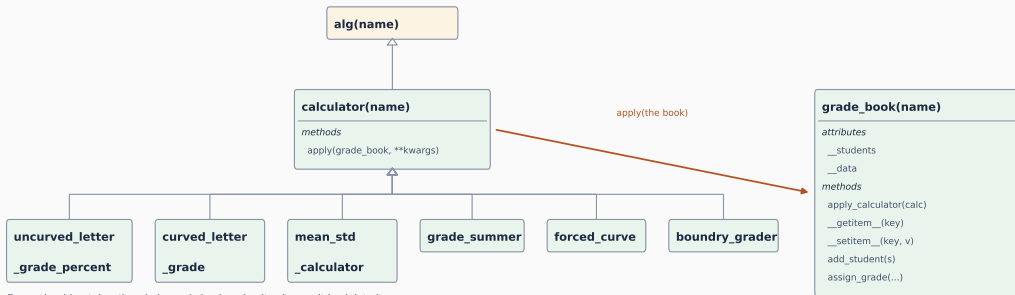
New kinds are now cheap

- `forced_curve` — force a target distribution
- `boundry_grader` — take the boundaries as an argument instead of hard-coding them
- Neither one touches `grade_book`
- That's the test of a good interface: extension without modification

→ cells 75, 76, 78, 79, 81

Where we ended up

One interface, and a blackboard to write on



Every algorithm takes the whole grade book and writes its result back into it, so the next one can pick it up. Adding a kind of calculation adds a class, nothing else.

Looking back

What this chapter was really about

- We wrote the gradebook three times, and each version taught the next
- Design iteration is **normal**. First versions are supposed to be wrong
- What you protect is the interface; implementations are cheap to replace
- Draw it first: Lab 6 asks for the UML before the blackjack code
- Next: enough software design — we turn to looking at data